

✓ Sentiment Analysis on IMDB Reviews using LSTM

Steps

1. Load the dataset
2. Clean Dataset
3. Encode Sentiments
4. Split Dataset
5. Tokenize and Pad/Truncate Reviews
6. Build Architecture/Model
7. Train and Test

```
import re # Regular expressions for text cleaning
import numpy as np # For mathematical operations and array handling
import pandas as pd # To load and manipulate the dataset
from nltk.corpus import stopwords # To get a collection of stop words (e.g., 'the', 'is')
from sklearn.model_selection import (
    train_test_split,
) # For splitting dataset into train/test sets
from tensorflow.keras.callbacks import ModelCheckpoint # To save the best model weights
from tensorflow.keras.layers import (
    Dense,
    Embedding,
    LSTM,
) # Neural network layers
from tensorflow.keras.models import Sequential # To initialize sequential model architecture
from tensorflow.keras.models import load_model # To load saved models from disk
from tensorflow.keras.preprocessing.sequence import (
    pad_sequences,
) # For sequence padding/truncating
from tensorflow.keras.preprocessing.text import (
    Tokenizer,
) # To encode text tokens into integers
```

```
# Load the IMDB dataset from CSV file
data = pd.read_csv("IMDB Dataset.csv")

# Display the raw dataframe content and shape
print(data)
```

```
              review sentiment
0    One of the other reviewers has mentioned that ... positive
1    A wonderful little production. <br /><br />The... positive
2    I thought this was a wonderful way to spend ti... positive
3    Basically there's a family where a little boy ... negative
4    Petter Mattei's "Love in the Time of Money" is... positive
...      ...      ...
49995 I thought this movie did a down right good job... positive
49996 Bad plot, bad dialogue, bad acting, idiotic di... negative
49997 I am a Catholic taught in parochial elementary... negative
49998 I'm going to have to disagree with the previou... negative
49999 No one expects the Star Trek movies to be high... negative
```

```
[50000 rows x 2 columns]
```

```
# Initialize the set of English stop words to filter out common insignificant words
import nltk

nltk.download("stopwords")
english_stops = set(stopwords.words("english"))
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

```
def load_dataset():
    df = pd.read_csv("IMDB Dataset.csv")
    x_data = df["review"] # Extract reviews as input features
    y_data = df["sentiment"] # Extract sentiments as target labels

    # PRE-PROCESS REVIEW TEXT
```

```

x_data = x_data.replace(
    {"<.*?>": ""}, regex=True
) # Remove HTML tags from reviews
x_data = x_data.replace(
    {"^[A-Za-z]": " "}, regex=True
) # Remove non-alphabet characters
x_data = x_data.apply(
    lambda review: [w for w in review.split() if w not in english_stops]
) # Remove stop words
x_data = x_data.apply(
    lambda review: [w.lower() for w in review]
) # Normalize text to lowercase

# ENCODE SENTIMENT LABELS -> Map text labels to binary values (1 & 0)
y_data = y_data.replace("positive", 1)
y_data = y_data.replace("negative", 0)

return x_data, y_data

```

```
# Execute the preprocessing pipeline and inspect outputs
```

```
x_data, y_data = load_dataset()
```

```

print("Reviews")
print(x_data, "\n")
print("Sentiment")
print(y_data)

```

Reviews

```

0      [one, reviewers, mentioned, watching, oz, epis...
1      [a, wonderful, little, production, the, filmin...
2      [i, thought, wonderful, way, spend, time, hot,...
3      [basically, family, little, boy, jake, thinks,...
4      [petter, mattei, love, time, money, visually, ...
...
49995  [i, thought, movie, right, good, job, it, crea...
49996  [bad, plot, bad, dialogue, bad, acting, idioti...
49997  [i, catholic, taught, parochial, elementary, s...
49998  [i, going, disagree, previous, comment, side, ...
49999  [no, one, expects, star, trek, movies, high, a...
Name: review, Length: 50000, dtype: object

```

Sentiment

```

0      1
1      1
2      1
3      0
4      1
..
49995  1
49996  0
49997  0
49998  0
49999  0

```

Name: sentiment, Length: 50000, dtype: int64

/tmp/ipykernel_6857/970979404.py:22: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. Please use `y\_data = y\_data.replace("negative", 0)` instead.

```
# Split data into 80% training set and 20% testing set
```

```

x_train, x_test, y_train, y_test = train_test_split(
    x_data, y_data, test_size=0.2
)

```

```

print("Train Set")
print(x_train, "\n")
print(x_test, "\n")
print("Test Set")
print(y_train, "\n")
print(y_test)

```

Train Set

```

33305  [i, somehow, managed, make, way, movie, dumbfo...
23860  [while, d, animation, highlight, show, job, we...
11367  [the, violent, death, fernando, ramos, da, sil...
25502  [i, usually, steer, clear, tv, movies, many, w...
48016  [i, swore, i, would, never, allow, devolve, bo...
...
48545  [the, th, floor, decidedly, mediocre, film, st...
42836  [i, recently, watched, ed, wood, jr, autobiogr...

```

```

6875      [i, really, looking, forward, show, given, qua...
3252      [when, na, young, eddie, hatch, window, dresse...
47375     [unlike, bond, detective, movies, alfred, hitc...
Name: review, Length: 40000, dtype: object

30109     [when, one, thinks, science, fiction, films, o...
31752     [when, hey, arnold, first, came, air, i, watch...
29559     [or, least, forceable, retirement, this, movie...
54        [the, percent, nations, nitwits, still, suppor...
49935     [nurse, betty, kind, movie, describe, poster, ...
...
22503     [amazing, astounding, brilliant, superb, those...
26860     [i, glad, i, saw, film, mr, sandler, became, f...
37101     [acclaimed, japanese, director, takashi, miike...
28204     [one, sit, utter, amazement, mess, film, amuse...
41093     [think, extremely, low, rent, version, heather...
Name: review, Length: 10000, dtype: object

```

Test Set

```

33305     0
23860     0
11367     1
25502     1
48016     1
..
48545     0
42836     0
6875      0
3252      0
47375     1
Name: sentiment, Length: 40000, dtype: int64

```

```

30109     1
31752     0
29559     0
54        0
49935     1
..
22503     1
26860     0
37101     0
28204     0
41093     0
Name: sentiment, Length: 10000, dtype: int64

```

Function to calculate the ceiling of the mean review length for padding configuration

```

def get_max_length():
    review_length = []
    for review in x_train:
        review_length.append(len(review))

    return int(np.ceil(np.mean(review_length)))

```

ENCODE REVIEW TEXT INTO INTEGER SEQUENCES

```

token = Tokenizer(
    lower=False
) # lower=False since data was already lowered in load_dataset()
token.fit_on_texts(x_train)
x_train = token.texts_to_sequences(x_train)
x_test = token.texts_to_sequences(x_test)

# Determine maximum sequence length limit
max_length = get_max_length()

# Pad sequences uniformly with post padding and truncation
x_train = pad_sequences(
    x_train, maxlen=max_length, padding="post", truncating="post"
)
x_test = pad_sequences(
    x_test, maxlen=max_length, padding="post", truncating="post"
)

# Calculate total vocabulary size plus one for padding token index 0
total_words = len(token.word_index) + 1

print("Encoded X Train\n", x_train, "\n")
print("Encoded X Test\n", x_test, "\n")
print("Maximum review length: ", max_length)

```

```
Encoded X Train
[[ 1 746 1213 ... 0 0 0]
 [375 797 734 ... 0 0 0]
 [ 2 1021 232 ... 1029 7 7149]
 ...
 [ 1 13 163 ... 73 626 517]
 [169 8827 95 ... 291 111 2999]
 [948 1163 1283 ... 0 0 0]]
```

```
Encoded X Test
[[ 169 5 1192 ... 22363 35081 165]
 [ 169 1263 3295 ... 0 0 0]
 [ 841 130 7527 ... 31 1 66]
 ...
 [5209 856 68 ... 0 0 0]
 [ 5 740 2010 ... 0 0 0]
 [ 31 480 259 ... 136 983 2776]]
```

Maximum review length: 130

```
# DEFINE MODEL HYPERPARAMETERS AND ARCHITECTURE
```

```
EMBED_DIM = 32
```

```
LSTM_OUT = 64
```

```
model = Sequential()
model.add(Embedding(total_words, EMBED_DIM, input_length=max_length))
model.add(LSTM(LSTM_OUT))
model.add(Dense(1, activation="sigmoid")) # Output layer with sigmoid for binary classification
model.compile(
    optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"]
)
```

```
print(model.summary())
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated
warnings.warn(
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

```
Total params: 0 (0.00 B)
```

```
Trainable params: 0 (0.00 B)
```

```
Non-trainable params: 0 (0.00 B)
```

```
None
```

```
# Configure ModelCheckpoint to save only the best weights based on training accuracy
```

```
checkpoint = ModelCheckpoint(
    "models/LSTM.h5", monitor="accuracy", save_best_only=True, verbose=1
)
```

```
# Train the LSTM neural network on the training dataset for 5 epochs
```

```
model.fit(
    x_train, y_train, batch_size=128, epochs=5, callbacks=[checkpoint]
)
```

```
Epoch 1/5
```

```
313/313 ————— 0s 241ms/step - accuracy: 0.5432 - loss: 0.6819
```

```
Epoch 1: accuracy improved from None to 0.57700, saving model to models/LSTM.h5
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
```

```
Epoch 1: finished saving model to models/LSTM.h5
```

```
313/313 ————— 79s 242ms/step - accuracy: 0.5770 - loss: 0.6683
```

```
Epoch 2/5
```

```
313/313 ————— 0s 247ms/step - accuracy: 0.5913 - loss: 0.6608
```

```
Epoch 2: accuracy improved from 0.57700 to 0.60663, saving model to models/LSTM.h5
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
```

```
Epoch 2: finished saving model to models/LSTM.h5
```

```
313/313 ————— 78s 248ms/step - accuracy: 0.6066 - loss: 0.6580
```

```
Epoch 3/5
```

```
313/313 ————— 0s 246ms/step - accuracy: 0.5839 - loss: 0.6568
```

```
Epoch 3: accuracy improved from 0.60663 to 0.66490, saving model to models/LSTM.h5
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
```

```
Epoch 3: finished saving model to models/LSTM.h5
313/313 ————— 77s 247ms/step - accuracy: 0.6649 - loss: 0.6009
Epoch 4/5
313/313 ————— 0s 237ms/step - accuracy: 0.8237 - loss: 0.4574
Epoch 4: accuracy improved from 0.66490 to 0.82270, saving model to models/LSTM.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format

Epoch 4: finished saving model to models/LSTM.h5
313/313 ————— 75s 238ms/step - accuracy: 0.8227 - loss: 0.4537
Epoch 5/5
313/313 ————— 0s 232ms/step - accuracy: 0.8457 - loss: 0.4105
Epoch 5: accuracy improved from 0.82270 to 0.84478, saving model to models/LSTM.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format

Epoch 5: finished saving model to models/LSTM.h5
313/313 ————— 73s 232ms/step - accuracy: 0.8448 - loss: 0.4144
<keras.src.callbacks.history.History at 0x7d40939712b0>
```

```
# Cell 10: Evaluate Model on Test Set (Fixed for newer TensorFlow versions)
# model.predict_classes was deprecated in newer TensorFlow/Keras versions; use predict and threshold instead
y_pred = (model.predict(x_test, batch_size=128) >= 0.5).astype(int).flatten()

true = 0
for i, y in enumerate(y_test):
    if y == y_pred[i]:
        true += 1

print("Correct Prediction: {}".format(true))
print("Wrong Prediction: {}".format(len(y_pred) - true))
print("Accuracy: {}".format(true / len(y_pred) * 100))
```

```
79/79  6s 72ms/step
Correct Prediction: 8055
Wrong Prediction: 1945
Accuracy: 80.55
```

```
# Load the pre-trained and saved model weights from disk
loaded_model = load_model("models/LSTM.h5")
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until

```
# Prompt the user to input a custom review string
review = str(input("Movie Review: "))
```

Movie Review: good

```
# Clean input text by removing punctuation and stop words
regex = re.compile(r"[^a-zA-Z\s]")
review = regex.sub("", review)
print("Cleaned: ", review)

words = review.split(" ")
filtered = [w for w in words if w not in english_stops]
filtered = " ".join(filtered)
filtered = [filtered.lower()]

print("Filtered: ", filtered)
```

```
Cleaned:  good
Filtered:  ['good']
```

```
# Convert the filtered custom review into token sequences and pad it
tokenize_words = token.texts_to_sequences(filtered)
tokenize_words = pad_sequences(
    tokenize_words, maxlen=max_length, padding="post", truncating="post"
)
print(tokenize_words)
```

[illegible]

```
# Feed the processed custom review sequence into the loaded model for prediction
result = loaded_model.predict(tokenize_words)
print(result)
```

1/1  1s 584ms/step
[[0.75912726]]

```
# Map the model output probability score to a final human-readable sentiment label
if result >= 0.7:
    print("positive")
else:
    print("negative")
```

positive